

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Constraint Based Problem Solving
Weightage:	40%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	RAGAVIS	Reg. No.:	192565027
Section / Batch:	SLOT – A (2025-29)	Date:	26-08-2026

Instructions to Students

- Identify the relevant concepts of I/O interfaces, interrupts, DMA, bus arbitration, and high-speed communication protocols.
- Analyze the I/O requirements considering CPU utilization, latency, throughput, bandwidth, and device priorities.
- Apply suitable I/O techniques such as DMA, vectored interrupts, memory-mapped I/O, nested interrupts, and bus arbitration.
- Compute performance measures such as data transfer rate, interrupt latency, CPU utilization, throughput, and transfer time.
- Interpret the results by evaluating CPU efficiency, communication performance, interrupt response, and suitability of the proposed I/O architecture.

QUESTIONS

1. Design an I/O communication mechanism for a smart agriculture system with the following constraints:
 - 50 field sensors transmit readings at different intervals.
 - Soil-moisture alerts must be processed within 20 μ s.
 - Use interrupt-driven I/O for critical events.
 - Use DMA for periodic bulk sensor-data transfer.
2. Construct an I/O subsystem for a video surveillance system with the following constraints:
 - 8 cameras continuously generate video streams.
 - The aggregate data rate is 1.5 GB/s.
 - Minimize CPU involvement during video transfer.
 - Use DMA with buffer management to prevent data loss.
3. Design a peripheral communication architecture for an industrial automation system with the following constraints:
 - 12 peripherals operate at different data rates.
 - High-priority control devices require deterministic response.
 - Low-priority devices must not monopolize the communication bus.
 - Implement priority-based bus arbitration.
4. Construct a data transfer mechanism for a real-time audio processing system with the following constraints:
 - Audio data must be transferred continuously at 200 MB/s.
 - Transfer interruptions must be minimized.
 - Use double buffering for continuous data processing.
 - Generate an interrupt only when a buffer transfer is completed.
5. Design an interrupt management system for a multicore embedded processor with the following constraints:
 - Support multiple interrupt sources simultaneously.
 - Distribute interrupts across processor cores to balance the workload.
 - Critical interrupts must have higher priority than routine interrupts.
 - Prevent excessive interrupt overhead during high system activity.

Code of Conduct

I RAGAVIS / 192565027 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: RAGAVIS

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Q. No	Problem Understanding (25)	Constraint Analysis (25)	Solution Development (25)	Application of Concepts (15)	Justification (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/25	/ 25	/ 15	/ 10	/ 100

Course Coordinator

Faculty Incharge

Question 1: I/O Communication Mechanism for a Smart Agriculture System

Given Constraints

- 50 field sensors transmitting readings at different intervals
- Soil-moisture alerts must be processed within 20 μ s
- Interrupt-driven I/O required for critical events
- DMA required for periodic bulk sensor-data transfer

1. Problem Understanding

The system generates two fundamentally different traffic classes: (a) rare, time-critical soil-moisture alerts that demand a hard real-time response, and (b) routine, high-volume periodic telemetry from 50 sensors that is latency-tolerant but must not burden the CPU. The core design problem is therefore to build a hybrid I/O subsystem that gives the alert path a guaranteed low-latency channel while offloading the bulk path entirely from the CPU.

2. Constraint Analysis

- 20 μ s latency bound is the dominant constraint — it rules out polling (variable, unbounded latency) for alerts and mandates interrupt-driven I/O with a short, deterministic interrupt-service path.
- 50 sensors sharing an address space requires structured addressing — memory-mapped I/O gives each sensor a unique register address without a separate I/O instruction set.
- Bulk periodic transfer is high in device count but low in per-transfer urgency, so CPU cycles must not be spent per sensor — this is a textbook DMA offload case.
- The two paths must be prioritised: alert interrupts must always pre-empt the routine DMA sweep.

3. Solution Development (Proposed I/O Architecture)

A hybrid, priority-partitioned I/O architecture is proposed:

- All 50 sensors are attached to a Sensor Interface Controller (SIC) and mapped into the CPU's memory address space (memory-mapped I/O) so each sensor's data/status register is directly addressable.
- Soil-moisture sensors are wired to a dedicated, highest-priority vectored interrupt line on the interrupt controller (PIC/APIC). Each alert type is assigned its own interrupt vector so the ISR is dispatched directly, with no cause-polling.

- The alert ISR is kept minimal: read threshold register → compare → set actuator/alarm flag → return, so total ISR execution stays short.
- A periodic hardware timer triggers a DMA controller in block-transfer mode, which sweeps all 50 sensor registers into a RAM buffer with zero CPU involvement, raising only one interrupt after the whole block completes.
- Interrupt priority is fixed so that the DMA-completion interrupt can be pre-empted by an incoming alert interrupt (nested/vectored priority).

4. Application of Concepts

- Memory-mapped I/O for uniform, address-based access to 50 sensors
- Vectored interrupt handling with dedicated priority for critical alerts
- Cycle-stealing / block-mode DMA for periodic bulk transfer
- Interrupt priority nesting to protect the 20 μ s deadline from bulk-transfer activity

5. Performance Computation

Interrupt-latency stage	Time (μ s)
Interrupt recognition (worst case, after current instruction)	2
Context save (registers/PC)	3
Vector fetch + ISR entry	2
ISR execution (read + threshold check + flag set)	8
Total worst-case latency	15

Budget = 20 μ s; computed worst-case path = 15 μ s, leaving a 5 μ s margin — the design meets the deadline.

6. Justification / Interpretation

Separating the traffic into an interrupt-driven critical path and a DMA-driven bulk path means each class is handled by the mechanism best suited to it: interrupts give the alert path a bounded, prioritised response (15 μ s < 20 μ s budget), while DMA removes the CPU entirely from the high-device-count, low-urgency telemetry sweep. Because the two paths share the same interrupt controller with strict priority ordering, an alert can always pre-empt an in-progress DMA-completion handler, so the real-time constraint is preserved even under simultaneous activity.

Question 2: I/O Subsystem for a Video Surveillance System

Given Constraints

- 8 cameras continuously generating video streams

- Aggregate data rate of 1.5 GB/s
- CPU involvement during video transfer must be minimised
- DMA with buffer management required to prevent data loss

1. Problem Understanding

This is a sustained, high-throughput streaming I/O problem, not a bursty or event-driven one. The design must move ~1.5 GB/s continuously from 8 independent camera sources into memory with negligible CPU load, while absorbing any short-term mismatch between capture rate and consumption (storage/encoding) rate so that no frames are ever dropped.

2. Constraint Analysis

- Per-camera rate = $1.5 \text{ GB/s} \div 8 = 187.5 \text{ MB/s}$ — far too high for programmed I/O or cycle-stealing DMA on a single shared channel; burst/block-mode DMA on multiple channels is required.
- "Minimise CPU involvement" rules out per-frame or per-packet interrupts — the CPU should only be signalled at coarse buffer boundaries.
- "Prevent data loss" means the design must tolerate the consumer (encoder/storage) being briefly slower than the producer (camera) — this is a buffering/synchronisation constraint, not a raw-bandwidth constraint alone.
- The interconnect (system bus) must itself provide more than 1.5 GB/s of sustained bandwidth, or the whole design is infeasible regardless of DMA scheme.

3. Solution Development (Proposed I/O Architecture)

A multi-channel burst-mode DMA architecture with ring buffering is proposed:

- A multi-channel DMA controller with 8 independent channels (one per camera) operates in burst/block transfer mode, moving data directly from each camera's capture interface to system RAM.
- Each channel is fed by a circular (ring) buffer implemented as multiple fixed-size segments (at least double, preferably triple buffering): DMA fills one segment while the video pipeline drains a previously-filled segment, so producer and consumer never contend for the same memory.
- The system bus is chosen with enough sustained bandwidth headroom above 1.5 GB/s (e.g., a PCIe link, see computation) so no channel is ever bandwidth-starved by another.
- The CPU receives only one interrupt per camera per completed buffer segment (not per frame), whose ISR simply hands off the filled buffer's pointer to the processing/encoding software and re-arms the next DMA descriptor (scatter-gather).

4. Application of Concepts

- Multi-channel, burst/block-mode DMA transfer
- Scatter-gather DMA descriptors for continuous re-arming without CPU polling
- Ring / multiple buffering for producer–consumer rate mismatch protection
- Bus-bandwidth sizing (PCIe) and interrupt coalescing at buffer-boundary granularity

5. Performance Computation

Parameter	Value
Per-camera data rate	187.5 MB/s
Chosen per-channel buffer size	32 MB
Buffer fill time ($32 \text{ MB} \div 187.5 \text{ MB/s}$)	$\approx 170 \text{ ms}$
Interrupts per camera per second	≈ 5.9
Total interrupts/sec for 8 cameras	≈ 47
Required sustained bus bandwidth	1.5 GB/s
PCIe 3.0 x4 effective bandwidth (reference)	$\approx 3.94 \text{ GB/s}$

With a 32 MB buffer per channel, total CPU interrupt load is only ~ 47 interrupts/sec for the entire 1.5 GB/s stream, and a PCIe 3.0 x4 link provides more than $2.5 \times$ the required bandwidth as headroom.

6. Justification / Interpretation

Multi-channel burst DMA satisfies the throughput requirement while the ring-buffering scheme guarantees that a slow consumer never causes an overwrite of unread data or a stall of the producer, directly preventing data loss. Because the CPU is interrupted only ~ 47 times per second in total rather than per-frame, CPU involvement is reduced to near zero, and the bus-bandwidth calculation confirms the chosen interconnect can sustain the aggregate 1.5 GB/s with comfortable margin.

Question 3: Peripheral Communication Architecture for an Industrial Automation System

Given Constraints

- 12 peripherals operating at different data rates
- High-priority control devices require deterministic response
- Low-priority devices must not monopolise the communication bus

- Priority-based bus arbitration must be implemented

1. Problem Understanding

The problem is a shared-bus arbitration design under mixed criticality: some peripherals control real-time actuators/sensors and must receive a bounded, predictable worst-case access time, while the remaining peripherals are less urgent but still need fair, guaranteed access so none of them is starved and none of them can hog the bus for an unbounded time.

2. Constraint Analysis

- Pure fixed-priority arbitration (e.g., simple daisy-chaining) gives determinism to high-priority devices but can starve low-priority devices indefinitely if high-priority traffic is frequent.
- Pure round-robin gives fairness but cannot guarantee the bounded, deterministic latency that control peripherals need.
- Therefore the 12 devices must be split into criticality tiers, and within the low-priority tier, each bus grant must be time-bounded so no single device can monopolise the bus.
- The arbitration logic itself (priority encoder) must be fast enough that its own decision time does not erode the determinism guarantee.

3. Solution Development (Proposed I/O Architecture)

A two-tier, time-sliced priority arbitration scheme is proposed:

- Peripherals are grouped into Tier 1 (deterministic control devices) and Tier 2 (remaining peripherals).
- A centralised bus arbiter built around a priority encoder always grants the bus to a pending Tier 1 request ahead of any Tier 2 request, giving control devices near-immediate, bounded access.
- Within Tier 2, access is arbitrated by round-robin so every low-priority device gets a turn in rotation, ensuring fairness.
- Every bus grant — Tier 1 or Tier 2 — is capped by a bus-grant timeout (a fixed maximum number of bus cycles); a device must release the bus at the timeout boundary even mid-burst, guaranteeing no device (in particular no Tier 2 device) can monopolise the bus.
- Because a Tier 2 grant is bounded in duration, the worst-case wait for a Tier 1 request is simply one Tier-2 timeout period, giving a computable deterministic bound.

4. Application of Concepts

- Priority-based bus arbitration using a priority encoder
- Two-tier criticality classification (fixed-priority + round-robin hybrid)
- Bus-grant time-slicing / watchdog timeout to bound transfer duration
- Worst-case latency (determinism) analysis for real-time control access

5. Performance Computation

Parameter	Value
Bus cycle time	100 ns
Max bus-grant slice (per device)	10 cycles = 1 μ s
Worst-case wait for a Tier 1 request (behind one in-progress Tier 2 slice)	$\leq 1 \mu$ s
Max devices in Tier 2 round-robin before repeat access	8 (illustrative split of 12)

The 1 μ s bound is deterministic because it depends only on the fixed slice length, not on how many Tier 2 devices are waiting.

6. Justification / Interpretation

The two-tier priority scheme gives Tier-1 control devices a bounded worst-case wait of just one bus-grant slice (computed as $\leq 1 \mu$ s), satisfying the deterministic-response requirement. Capping every grant with a timeout — including Tier-1 grants — guarantees that no device, and in particular no low-priority device, can occupy the bus indefinitely, directly satisfying the anti-monopolisation constraint while the internal round-robin among Tier 2 devices keeps their access fair.

Question 4: Data Transfer Mechanism for a Real-Time Audio Processing System

Given Constraints

- Audio data must be transferred continuously at 200 MB/s
- Transfer interruptions must be minimised
- Double buffering required for continuous data processing
- An interrupt must be generated only when a buffer transfer completes

1. Problem Understanding

The requirement is uninterrupted, continuous streaming transfer — any gap in the data flow corresponds to an audible glitch/underrun, so the architecture must guarantee that a transfer target is always ready before the previous one finishes, while keeping CPU interrupt overhead as low as reasonably possible.

2. Constraint Analysis

- A single buffer cannot be used because it would need to be emptied and refilled atomically, forcing a stall (interruption) at every cycle boundary — this directly motivates the double-buffering requirement.
- Buffer size creates a direct trade-off: a small buffer completes (and interrupts) very frequently, raising CPU/ISR overhead; a large buffer reduces interrupt frequency but increases the latency between capture and processing.
- The ISR itself must be minimal (buffer-pointer swap + signal only) so its execution time does not itself become a source of jitter or interruption.
- The sustained rate (200 MB/s) must be achievable by the chosen DMA mode without saturation, or gaps will appear regardless of buffering.

3. Solution Development (Proposed I/O Architecture)

A double-buffered (ping-pong), block-mode DMA transfer scheme is proposed:

- Two equal-sized buffers (A and B) are allocated in memory. The DMA controller is configured in block/ping-pong mode so that, on completion of buffer A, it automatically switches its target address to buffer B without CPU intervention, and vice versa.
- While DMA fills buffer B, the audio-processing routine drains/consumes buffer A, and the roles swap on the next completion — guaranteeing one buffer is always ready.
- An interrupt is generated only on each buffer-completion event (not per sample or per burst); the ISR does nothing but swap the active-buffer pointer and post a signal to the audio-processing task, then returns immediately.
- Buffer size is chosen to balance interrupt overhead against latency (see computation) rather than left arbitrary.

4. Application of Concepts

- Double buffering (ping-pong buffering) for gapless streaming
- Block-mode DMA with automatic buffer-address switching
- Minimal, buffer-boundary-only interrupt generation
- Buffer-size vs interrupt-overhead vs latency trade-off analysis

5. Performance Computation

Parameter	Value
Sustained transfer rate	200 MB/s
Chosen buffer size	1 MB
Time to fill one buffer ($1 \text{ MB} \div 200 \text{ MB/s}$)	5 ms
Interrupt rate	200 interrupts/sec
Latency introduced by buffering	$\approx 5 \text{ ms}$

A 1 MB buffer keeps interrupt load to 200/sec (low CPU overhead) while adding only ~5 ms of pipeline latency, an acceptable trade-off for continuous embedded audio processing; a smaller buffer could be chosen if lower latency is prioritised, at the cost of a higher interrupt rate.

6. Justification / Interpretation

Ping-pong double buffering guarantees that a fully-prepared buffer is always available for consumption while the other is being filled, so the continuous 200 MB/s stream never has to stall — directly meeting the "minimise interruptions" requirement. Restricting interrupts to buffer-completion boundaries (200/sec, each handled by a trivial ISR) keeps CPU overhead low without compromising the ability of the system to respond in time to refill the next buffer.

Question 5: Interrupt Management System for a Multicore Embedded Processor

Given Constraints

- Must support multiple interrupt sources simultaneously
- Interrupts must be distributed across processor cores to balance workload
- Critical interrupts must have higher priority than routine interrupts
- Excessive interrupt overhead during high system activity must be prevented

1. Problem Understanding

The problem extends single-core interrupt handling into a multicore setting: the design must (1) accept and correctly prioritise interrupts arriving from many simultaneous sources, (2) route them so that no single core becomes an interrupt-handling bottleneck, and (3) remain efficient — i.e., not let interrupt-handling overhead itself dominate the CPU — when interrupt activity is high.

2. Constraint Analysis

- Simultaneous multi-source interrupts require a controller capable of queuing/prioritising several pending requests at once, not a single-line interrupt scheme.
- Load balancing across cores implies interrupts cannot be hard-wired to one fixed core; routing must be configurable/dynamic (affinity or lowest-priority-core delivery).
- Critical-vs-routine priority implies a nested/pre-emptive interrupt model: a routine ISR executing on a core must be pre-emptible by a critical interrupt.
- "Excessive overhead at high activity" identifies interrupt-storm risk: at very high rates, the fixed per-interrupt entry/exit cost (context save/restore, vector dispatch) can consume most of the CPU — this must be mitigated for non-critical sources without hiding critical events.

3. Solution Development (Proposed I/O Architecture)

A distributed, priority-nested interrupt architecture with selective coalescing is proposed:

- Each core has a Local Interrupt Controller (Local APIC-style), and a global I/O interrupt controller collects all external interrupt sources and routes each one to a specific core using a load-balancing delivery policy (e.g., round-robin or lowest-priority-core-first).
- Every interrupt source is assigned to one of two priority classes: Critical and Routine, each with its own vector/priority level in the local controller's priority register.
- The controller supports nested interrupts: a Critical interrupt is allowed to pre-empt a currently executing Routine ISR on the same core (via priority masking), guaranteeing critical events are never delayed behind routine work.
- For high-frequency Routine sources, interrupt coalescing/batching is applied — multiple routine events are accumulated and reported as a single interrupt once a count or time threshold is reached — while Critical sources always bypass coalescing and interrupt immediately.
- A system-load threshold can additionally mask/rate-limit low-priority interrupt sources when overall activity is very high, protecting the CPU's ability to service critical work.

4. Application of Concepts

- Distributed (Local/Global APIC-style) multicore interrupt controller architecture
- Vectored, nested/pre-emptive interrupt priority levels (critical vs routine)
- Interrupt affinity / load-balanced routing across cores

- Interrupt coalescing and priority-based masking to control overhead under high load

5. Performance Computation

Scenario	Overhead
Routine interrupt rate (no coalescing)	100,000 /sec
Per-interrupt entry/exit overhead	1 μ s
CPU time consumed (no coalescing)	100 ms/sec = 10% of one core
Coalescing batch factor	20 events/interrupt
Effective interrupt rate (with coalescing)	5,000 /sec
CPU time consumed (with coalescing)	5 ms/sec = 0.5% of one core

Coalescing routine interrupts by a factor of 20 cuts CPU overhead from 10% to 0.5% of a core — a $20\times$ reduction — while critical interrupts remain uncoalesced and immediate.

6. Justification / Interpretation

Distributing interrupts across per-core local controllers under a global router balances load so no single core saturates under many simultaneous sources, satisfying the workload-balancing requirement. Nested priority levels ensure critical interrupts always pre-empt routine ones, satisfying the priority requirement, and the computation shows that selective coalescing of routine interrupts reduces CPU overhead by roughly $20\times$ at high activity levels without ever delaying a critical event, directly satisfying the overhead-prevention constraint.